



Angular Developer's Guide

A Quick Reference Guide for Programmers
Building UI Projects with Angular

Author: Mustaq Basha Shaik

Table of Content

• Introduction	01
• General Software Principles	01
• What is Single Page Application (SPA)	01
• Using the Angular CLI	02
• Feature Development	02
• Separating Business Logic from View	03
• Handling Errors on the Client Side	04
• Using Component Bindings	05
• Make Form Controls Reactive	06
• Routing	07
• Testing	08
• Containerizing Angular apps	09
• Resources	12

Introduction

Purpose of this document

This guide is for programmers who are building UI projects with Angular. For UI developers with non-CS backgrounds, this guide provides links to a few external resources that describe best practices of Object-Oriented Programming.

Before You Begin

- Develop a working knowledge of Angular (e.g. By viewing the [Angular tutorial](#)).
- Skim the Fundamentals and Techniques sections of the Angular docs.
- Review the [Angular style guide](#).
- Learn about [ReactiveX](#) and its JavaScript implementation - [RxJS](#).
- Learn about [component architectures for Angular's reactive forms](#).

General Software Principles

- Build Angular apps using SOLID principles:
[https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design))
- Build Angular apps using the 12-factor app pattern:
<https://12factor.net/>

For specific guidance on how to manage your app's configuration, refer the [Containerizing Angular Apps](#) chapter of this guide.

What is a Single Page Application (SPA)?

A Single Page Application (SPA) is a web application used to create fast, responsive, and easy-to-maintain websites. Single Page Applications are web applications that load a single HTML page and dynamically update that page as the user interacts with the app.

Single Page Application loads a single file and builds on that singular file as the user traverses through the app. Single Page Applications use AJAX and HTML5 to create fluid and responsive web apps without constant page reloads. However, this means much of the work happens on the client side i.e. in JavaScript.

Benefits of Single Page Application

Apart from making JavaScript coding a little wiser for developers, Single Page Applications have two primary benefits for users.

- The first benefit is eliminating that annoying page change when you click on a link. In an Single Page Application, the navigation controls and main interface typically stay on the page when you click a link; only the piece of content you want changed actually gets changed.

- The second benefit is the speed boost that you get from a lighter server response payload for the chunk of content vs an entire page. The lighter payload transfers across the network faster and the browser can incorporate the new piece of content quickly rather than redrawing an entirely new page.

What's in it for you?

Faster and more efficient applications compared to conventional websites.

How is this possible? Well, for most conventional websites, entire pages are loaded and reloaded continuously which causes a slowdown, resulting in a frustrating user experience and places unnecessary strain on the server. Single Page Application quickly solve this problem by only loading small entities of a webpage at a time. By utilizing Single Page Application, you are essentially decreasing the server load and increasing site speed for a better user experience.

Angular is a powerful JavaScript framework for building dynamic web applications, which is popular nowadays. The great thing about Angular is that it has a set of ready-to-use modules to simplify building of single page applications (SPAs).

Using the Angular CLI

Angular provides a command-line tool that scaffolds new apps, components, and new services. Additionally, it provides convenient access to a host of related tools:

<https://cli.angular.io/>

- To scaffold a new app, use the following command:

```
ng new my-app
```

Some server-side technologies, such as Grails and .NET, provide alternative ways to create a new Angular application. If you are using one of these technologies build these applications using a structure consistent with the CLI scaffolding. You should be able to run commands on the project using the CLI, even if you did not use the CLI to create the project.

Feature Development

Using SASS/SCSS

SASS is a more advanced UI formatting language that compiles into CSS. It eliminates repetition and enables code reuse in your CSS presentation markup. The following tutorial shows how to incorporate SASS into a new or existing app:

<https://scotch.io/tutorials/using-sass-with-the-angular-cli>

In summary

Creating a new app:

```
ng new my-sassy-app --style=scss
```

Modifying an existing app:

```
ng set defaults.styleExt scss
```

Either of these commands will add the **defaults.styleExt** setting to your project's **.angular-cli.json** file.

Using Materialize CSS

The UX team recommends using the [Materialize CSS](#) framework to layout pages.

Using Kendo UI for Angular

Telerik provides a library of ready-made [UI components](#) for Angular called the Kendo library.

To install the Kendo UI in your Angular project, follow the instructions in Telerik's Getting Started guide. To gain access to professional Telerik support and the Telerik desktop client, reach out to someone who already has a license. Currently, anyone who has a support account can add new users from the Telerik Support page.

Use Angular Material

Angular Material components provide reusable, well-tested, accessible UI components (similar to kendo) with a reference implementation of Google's material design specification. Further, for Angular 2 development, material components are not fully available for all UI components, e.g., Tree, etc. For the gaps in UI components that are not available alternative components can be picked from either Kendo or other UI frameworks.

Please check Kendo's Angular component's list [here](#).

Creating a UI Model

Create a Typescript object model that reflects the current state of your UI. Recognize that this model may differ from the models of your APIs. For example, if you are implementing a tabbed interface, your UI model should have a field for the currently selected tabs, flags for shown/hidden tabs, etc. Use services to map data from your API models to the client model.

Separating Business Logic from View

Your components should contain the least possible amount of code. When you need to perform business logic, parsing, processing, mapping, or REST operations, do these things in a service. This makes unit testing easier and reduces the complexity of your code.

Handling Errors on the Client Side

In an Angular app, one of the most common sources of errors are network failures or problems accessing the server. At any given time your app should be ready to make an HTTP request, your app should be able to handle potential errors. In some scenarios, it may make sense to implement some type of recovery procedure, such as retry. If recovery is not possible (or not implemented), the app should inform the user that there was a problem.

When implementing a service that makes HTTP requests, do not encapsulate error handling in the requesting service. The service method should return a Promise. Indicate failure by marking the Promise as rejected, and let the caller determine how to handle the failure. When implementing a service that makes HTTP requests, do not encapsulate error handling in the requesting service. The service method should return a Promise. Indicate failure by marking the Promise as rejected, and let the caller determine how to handle the failure.

Sample Service using HttpClient

```
import {Injectable} from '@angular/core'
import {Translation} from "../translation"
import {HttpClient} from "@angular/common/http"
import {ConfigService} from "../config.service"
import {Template} from "../template"

@Injectable()
export class TranslationsApiService {
  constructor(
    private configService: ConfigService,
    private httpClient: HttpClient) {
  }

  getTemplateByType(app: string, resourceType: string): Promise<Template>
  {
    let templatePromise =
    this.httpClient.get<Template>(`${this.configService.getConfig().url}/Templates/
    ${app}/${resourceType}`).toPromise()
      .catch(this.handleError.bind(this))
    return templatePromise
  }

  private handleError(e: any) {
    console.error('Error accessing the Translations service', e)
    throw e
  }

  saveUpdateTranslation(app: string, resourceType: string, ietfLanguageTag:
  string, resourceId: string, translation: Translation) {

    return
    this.httpClient.post<Translation>(`${this.configService.getConfig().url}
    /Translations/${app}/${resourceType}/${resourceId}/lan-
    guage/${ietfLanguageTag}`,
      translation).toPromise()
      .catch(this.handleError.bind(this))
  }
}
```

Sample form submission and error handling in Component code

```
onFormSubmit($event: Event): void {  
  /*  
   * If the form submit event does not have error handling, the  
   * default form submit behavior will kick in,  
   * triggering a page refresh.  
   */  
  try {  
    if (!this.questionEditTestForm.valid) {  
      console.log(`Form invalid: ${this.questionEditTestForm}`)  
      return  
    }  
  
    this.saveQuestion()  
  }  
  catch (e) {  
    this.toasterService.error('Error submitting the question defi-  
nition form.', e)  
  }  
}  
  
private saveQuestion() {  
  console.log('Saving translation', this.translation)  
  this.readMessagesFromForm()  
  this.questionTranslationService.saveTranslation(this.ques-  
tionDefinitionId.toString(), this.translation)  
    .then(() => {  
      console.log('Translation saved')  
      this.toasterService.info('Question definition saved!')  
    })  
    .catch((e: any) => {  
      this.toasterService.error("Error saving the translated  
text for a question.", e)  
    })  
}
```

Using Component Bindings

An Angular interface is composed of (potentially) reusable components, which each have a single responsibility. Components should accept input using model bindings, and publish output using event emitters.

Designing components for reuse

Even if you use a component only once in your app, design it in a way that facilitates reuse. A Component should prefer to control its child components using bindings, rather than stateful services. A component should never make assumptions about the context of its usage. It should not know whether its parent component uses it as part of a FormGroup or a FormArray. It should not have references to its parent components. (For example, do not inject parents using `@Host`).

A component should have its own styling. If you render the component on a page by itself, it should have at least some minimal, reasonable formatting.

Make Form Controls Reactive

If your app accepts and validates user input, build input components using the [Reactive Forms](#) technique. When you build a form component that references another component in its template, you are implicitly building a tree hierarchy of components.

```
{
  "shipClass": "Constitution",
  "designation": "NCC-1701",
  "shipSections": [
    {
      "sectionName": "Helm"
    },
    {
      "sectionName": "Engineering",
      "engineSettings": {
        "maxWarpSpeed": 9,
        "shieldsUp": true
      }
    }
  ]
}
```

Suppose I want to create a master-detail form where the user can enter this data. We recommend building a hierarchy of components, such that the UI model hierarchy and the component hierarchy match. Model each level of the hierarchy as a FormGroup.

Based on the example data, we would expect to see a top-level ship component, which has a FormGroup, two FormControls (shipClass, designation) and a repeater. Within the repeater, there should be a ShipSection component for each list item. Within the ShipSection component, we define a nested FormGroup with a FormControl called sectionName.

The second ship section has a child object called engine settings. Do not create complexity by implementing the engine settings controls in the ShipSection component. Instead, implement an EngineSettings component with another nested FormGroup.

Binding Nested Reactive Form Controls

FormControls and FormGroups need to be set up in a hierarchy. If this hierarchy spans multiple components, we need a way to connect the FormGroup elements in the parent with FormGroup elements in the children.

In the sample component below, we expose an @Output binding called formCreated. This event triggers after the component is initialized. We use the event handler in the parent component's template to connect the FormGroups for the parent and the child.

Sample Reactive Form Component

```
import {Component, EventEmitter, Input, OnInit, Output} from '@angular/core'
import {FormBuilder, FormGroup, Validators} from "@angular/forms"
import {QuestionMessages} from "../model/client/question-messages"

@Component({
  selector: 'app-question-translation-editor',
  templateUrl: './question-translation-editor.component.html',
  styleUrls: ['./question-translation-editor.component.scss']
})
export class QuestionTranslationEditorComponent implements OnInit {
  @Input() public messages: QuestionMessages
  @Output() public formCreated: EventEmitter<FormGroup> = new EventEmitter<FormGroup>(true)

  public questionMessagesForm: FormGroup

  constructor(private formBuilder: FormBuilder) {
  }

  ngOnInit() {
    this.questionMessagesForm = this.toFormGroup(this.messages)
    this.formCreated.emit(this.questionMessagesForm)
  }

  private toFormGroup(messages: QuestionMessages): FormGroup {
    return this.formBuilder.group({
      shortLabel: [messages.shortLabel, Validators.required],
      questionText: [messages.questionText, Validators.required],
      longDescription: messages.longDescription,
      footnote: messages.footnote,
      contextHelp: messages.contextHelp
    })
  }
}
```

Binding the component in the parent's template

```
<app-question-translation-editor *ngIf="translation"
  [messages]="translation.messages"
  (formCreated)="parentForm.addControl('messages', $event)">
</app-question-translation-editor>
```

Routing

When implementing navigation for your app, prefer to use the [router](#) provided by Angular. Each screen of your app should have its own route.

The router provides an intuitive browsing experience to end users. With router, you can make individual records and pages bookmark-able. Further, the router also provides sensible behavior for the back and forward buttons in a user's browser. Each main route in your app should have its own feature module. When implementing routes for your app, make sure to test what happens when a user loads a route from a bookmark or external link.

Testing

Build Angular apps using appropriate testing practices (TDD, BDD).

Unit testing

All Angular components, services, and classes should be unit-tested.

Testing Components that have children

When unit testing a class, you might want to isolate it from its dependencies. If you are testing a component that has nested child components, the recommended strategy is to replace the child components with mocked directive definitions in your TestBed configuration.

Sample TestBed with mock component

```
@Directive({selector: 'app-question-translation-editor'})
class MockQuestionTranslationEditorDirective {
  @Input() public messages: QuestionMessages
  @Output() public formCreated: EventEmitter<FormGroup> = new EventEmitter<FormGroup>()
}
beforeEach(async(() => {
  TestBed.configureTestingModule({
    imports: [ReactiveFormsModule, FormsModule],
    declarations: [QuestionEditTestComponent, MockQuestionTranslationEditorDirective],
    providers: [
      {provide: QuestionTranslationService, useValue: {}},
      {provide: ToasterService, useValue: {}}
    ]
  })
  .compileComponents()
}))
```

Sample test checking mock component's state

```
it('should set the question messages in the child component', fakeAsync(inject([FormBuilder, QuestionTranslationService],
  (formBuilder: FormBuilder, mockQuestionTranslationService: QuestionTranslationService) => {
    mockQuestionTranslationService.batchTranslate = crateSpy('batchTranslate').and.callFake(() => {
      return Promise.resolve([TEST_QUESTION_TRANSLATIONS[0]])
    })
    fixture.detectChanges()
    tick()
    fixture.detectChanges()

    let mockTaskElement = fixture.debugElement.query(By.directive(MockQuestionTranslationEditorDirective))
    expect(mockTaskElement).toBeTruthy('No element for this directive')
    let mockTaskComponent = mockTaskElement.injector.get(MockQuestionTranslationEditorDirective) as MockQuestionTranslationEditorDirective
    expect(mockTaskComponent).toBeTruthy('No editor component')
    expect(mockTaskComponent.messages).toEqual(TEST_QUESTION_TRANSLATIONS[0].messages)
    let formCreatedEvent = mockTaskComponent.formCreated
    expect(formCreatedEvent).toBeTruthy('No formCreatedEvent')
    let childForm = formBuilder.group({})
    formCreatedEvent.emit(childForm)
    expect(component.questionEditTestForm.get('messages')).toEqual(childForm)
  })))
```

Containerizing Angular apps

Building the *dist* folder

When deploying your Angular app, do not serve the app using Node. Node is not a production-ready application server. You can package up your application by running the 'build' NPM task:

```
npm run build --prod --build-optimizer
```

If need be, you can customize the 'build' task in the scripts section of your app's *package.json*:

```
"scripts": {  
  "ng": "ng",  
  "start": "ng serve --environment=dev",  
  "build": "ng build --environment=prod",  
  "test": "ng test --watch=false",  
  "lint": "ng lint",  
  "e2e": "ng e2e"  
},
```

Here, NPM is invoking the Angular CLI tool. The Angular CLI tool knows how to build and minify the app using Webpack. There is no need to have the CLI tool available in the PATH to run this build command; you simply need to have angular-cli installed in your devDependencies.

When the build is finished, the root directory of your Angular project will contain a dist folder. The dist folder contains HTML, JS, and resource files that you can serve up to the client in order to run your app.

Environment-specific Settings

In a real business application, you will want to deploy your Angular app in various environments (DEV, QA, UAT, PROD, etc.). Angular lets you define one or more environments for your project in your project source code, by creating a TypeScript file for each environment:

```
src/app/environments/  
  environment.ts  
  environment.prod.ts
```

You can specify the environment as a flag to the CLI when building the application, as shown above. You should NOT create a separate environment file for each deployment environment.

The Environment-in-code Anti-pattern

Recall the [12-Factor](#) App pattern:

Part III, Config says that we should store our configuration in our deployment environment. The recommended approach is to store environment-specific properties as system environment variables in our deployment environment. You should **NOT** include environment-specific properties in your source code.

"A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials."

Part V, Build, Release, Run says that we should separate our **Build** process from our **Release** process. If we set our environment in our NPM build, we are combining the **Build** and **Release** into a single step.

When we use Angular's built-in environment mechanism, we are combining build and release into a single step. This forces us to build a separate binary (a separate version of the dist folder) for each deployment environment. If a new check-in in our project's source repo introduces changes in between the time when we built DEV and the time when we build PROD, we could run into PROD issues.

Build Once - Run Anywhere

What we want to do is build a single binary (in this case, the dist folder) and **reuse the same binary** for each deployment. This prevents inconsistencies from creeping into our project in between deployments. We want the software artifact we deploy in PROD to be exactly the same as the one we tested in DEV and QA. We can think of this as a "build once, run anywhere" philosophy. You should only define two environments in your Angular project:

- **environment.ts** – Reasonable default settings for debugging on your local machine.
- **environment.prod.ts** – Production settings that your app will use when deployed to a container environment (e.g. Docker).

Because Angular is a client-side framework, it does not have any provisions to access environment variables on the HTTP server. We will introduce a mechanism to override the settings in `environment.prod.ts` at runtime using data read from the server.

Our solution will have two parts:

- A server-side capability that reads specific environment variables and serves them up as a web service.
- An Angular service that runs on startup, reads data from the server and modifies the Angular 'environment' properties.

Angular has built-in exception handling. This can be built on to with toastr messaging and sentry.io for a more robust client-side logging solution.

Set up for sentry.io is straightforward:

1. Use NPM to install the dependency

```
npm install raven-js --save
```

2. Set up your NgModule config to use sentry for automatic unhandled exception capture.

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule, ErrorHandler } from '@angular/core';
import { AppComponent } from './app.component';

import { environment } from '../environments/environment';
import * as Raven from 'raven-js';

Raven
  .config( 'https://<YOUR-SENTRY-KEY>@sentry.io<YOUR-PROJECT-ID>' )
  .install();

export class RavenErrorHandler implements ErrorHandler {
  handleError(err: any): void {
    Raven.captureException(err.originalError);
  }
}

export function provideErrorHandler() {
  if (environment.production) {
    return new RavenErrorHandler();
  } else {
    return new ErrorHandler();
  }
}

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [{ provide: ErrorHandler, useFactory: provideErrorHandler }],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Example explicit usage: `Raven.captureException(new Error('Message'));`

Conclusion:

With this eBook, I have covered the initial ground for you to start implementing Angular applications. Angular is a time-tested tool which makes building applications super easy. Not just that, Angular empowers the developers to build applications that live well on desktop, or mobile. Here we've explored the thought process behind solving many of the most common SPA and User Interface challenges that ubiquitously come up during the application development.

Resources

Build Once - Run Anywhere

<https://angular.io/>

Reactive Forms - <https://angular.io/guide/reactive-forms>

Angular Style Guide - <https://angular.io/guide/styleguide>

Angular Material - <https://material.angular.io/>

Materialize CSS

<http://materializecss.com/>

Kendo

Kendo UI for Angular - <https://www.telerik.com/kendo-angular-ui/>

Component demos - <https://www.telerik.com/kendo-angular-ui/components/>

NPM

<https://www.npmjs.com/>

Sentry.io

<https://docs.sentry.io>

Docker

<https://www.docker.com/>

About Evoke

Evoke Technologies is an innovative information technology services firm offering world-class software solutions. We are a client-centric and relationship-based company with a focus to provide cost-effective software development and support solutions. We are SEI-CMMI Level 3 appraised, ISO 27001 and 9001 compliant IT enterprise that focuses on quality and delivering business value. Incorporated in the year 2003, Evoke Technologies has been actively helping global corporations to innovate and transform their businesses utilizing IT. Our global delivery and engagement models help us to offer affordable long-term software solutions that meet our client's IT challenges and add business value. We constantly innovate and implement new software methodologies to provide our clients with a superior experience.

EVOKE TECHNOLOGIES



+1 (937) 660-4923



sales@evoketechnologies.com

www.evoketechnologies.com

7106 Corporate Way,
Dayton, OH - 45459,
United States



About the Author

Mustaq Basha Shaik, Practice Manager - UI/UX

Mustaq Basha has been associated with Evoke Technologies for 8 years now and carries over 16 years of experience in UI/UX and visual design. Basha has an undying passion for web designing and holds high-level expertise in various tools and technologies such as UI/UX/MEAN/MERN, Project Management. He considers his favorite tech guru as Bill Gates and in his spare time, enjoys exploring different genres of music, taking a thrilling trip into the wild and exercising regularly.

For more details please reach us at uipractice@evoketechnologies.com